Assignment 5 – Denoising Autoencoder and Classification Robustness

Angsar Shaumen Bekzat Sundetkhan

April 23, 2026

Abstract

We implement a fully-connected denoising autoencoder trained on clean sensor windows from the Jena Climate dataset, and evaluate its effect on the robustness of the 1-D CNN classifier from Assignment 4. Gaussian noise with five standard deviation levels is injected into test windows to simulate sensor corruption. We compare the CNN classifier's performance (accuracy, precision, recall, F1-score) on **clean**, **noisy**, and **autoencoder-denoised** variants. Results show that the autoencoder consistently recovers classification quality under moderate to high noise, recovering up to **+2.6% accuracy** and **+2.7% F1** at the highest tested noise level.

---

1. Introduction

Neural networks deployed in production environments must tolerate imperfect inputs caused by sensor failures, communication noise, and environmental interference. A common mitigation strategy is to place a denoising preprocessing module before the main model. Autoencoders, trained solely on clean data, learn the manifold of valid inputs and can project noisy observations back onto that manifold.

In this assignment we:

1. Build a fully-connected autoencoder with a 64-dimensional bottleneck and train it to reconstruct clean time-series windows.
2. Inject Gaussian noise at five levels ($\sigma \in \{0.05,\ 0.10,\ 0.20,\ 0.30,\ 0.50\}$) into the test set and measure the degradation of the CNN classifier.
3. Pass the noisy test data through the autoencoder and re-evaluate the classifier, quantifying the recovery.
4. Conduct a noise-level sweep to visualise the crossover point at which denoising becomes beneficial.

---

2. Dataset and Pre-processing

Dataset: Jena Climate dataset (Max Planck Institute for Biogeochemistry). The same pre-processed version used in Assignments 1-4 is reused here.

Property	Value
Total rows	420,533
Raw features	26 numeric
Target column	T (degC)
Binary threshold (median)	9.42 degC
Window size	24 timesteps (= 4 h)
Train / Val / Test split	70% / 15% / 15% (chronological)

All features were standardised with StandardScaler fitted **only on the training set** to prevent data leakage.

Noise injection: Gaussian noise $\mathcal{N}(0,\, \sigma^2)$ was added independently to every feature of every test window **after** standardisation, keeping noise in the normalised feature space:

$$\tilde{x} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

---

3. Autoencoder Architecture

The autoencoder is a **symmetric fully-connected** (dense) network. Input and output dimensions equal the flattened window: 24 steps $\times$ 26 features = 624.

Layer	Units	Activation
Input	624	—
Encoder Dense 1	256	ReLU
Encoder Dense 2	128	ReLU
Bottleneck	**64**	**ReLU**
Decoder Dense 1	128	ReLU
Decoder Dense 2	256	ReLU
Output	624	Linear

- **Loss:** Mean Squared Error (MSE)
- **Optimizer:** Adam ($lr = 10^{-3}$)
- **Batch size:** 1,024
- **Max epochs:** 30 with early stopping (patience = 5, monitoring val_loss)
- **Best training MSE:** 0.013910
- **Best validation MSE:** 0.013918

The near-identical train and validation loss confirms that the autoencoder generalises well without overfitting.

---

4. Experimental Results

4.1 Autoencoder Training

The model converged within approximately 15 epochs. Figure 1 shows that the training and validation loss track closely throughout training with no signs of overfitting. Early stopping triggered after the validation loss plateaued.

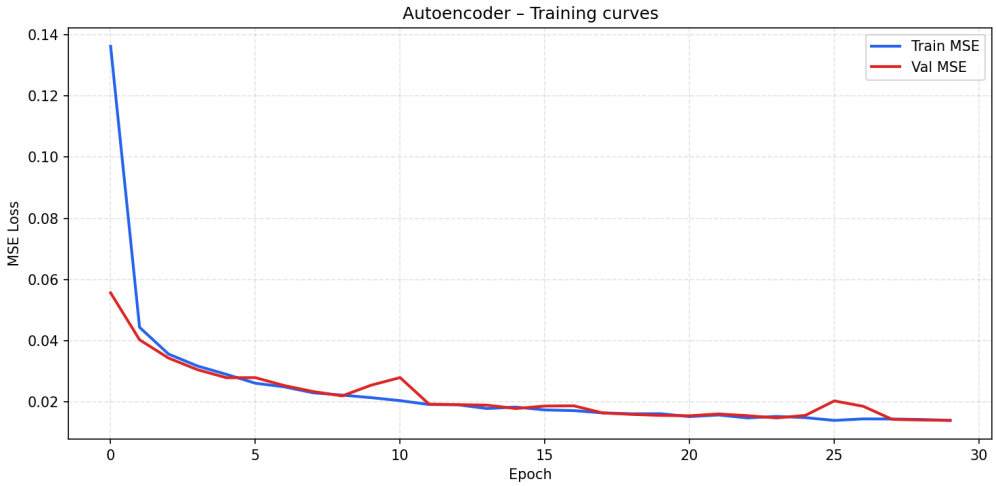

Figure 1: Autoencoder training and validation MSE loss over epochs. Both curves converge tightly, indicating good generalisation.

4.2 Signal Reconstruction

Visual inspection of randomly selected test windows (Figure 2) confirms that the denoised signal closely tracks the ground-truth clean signal while the raw noisy signal deviates noticeably. Reconstruction MSE on the test set: **0.013918**.

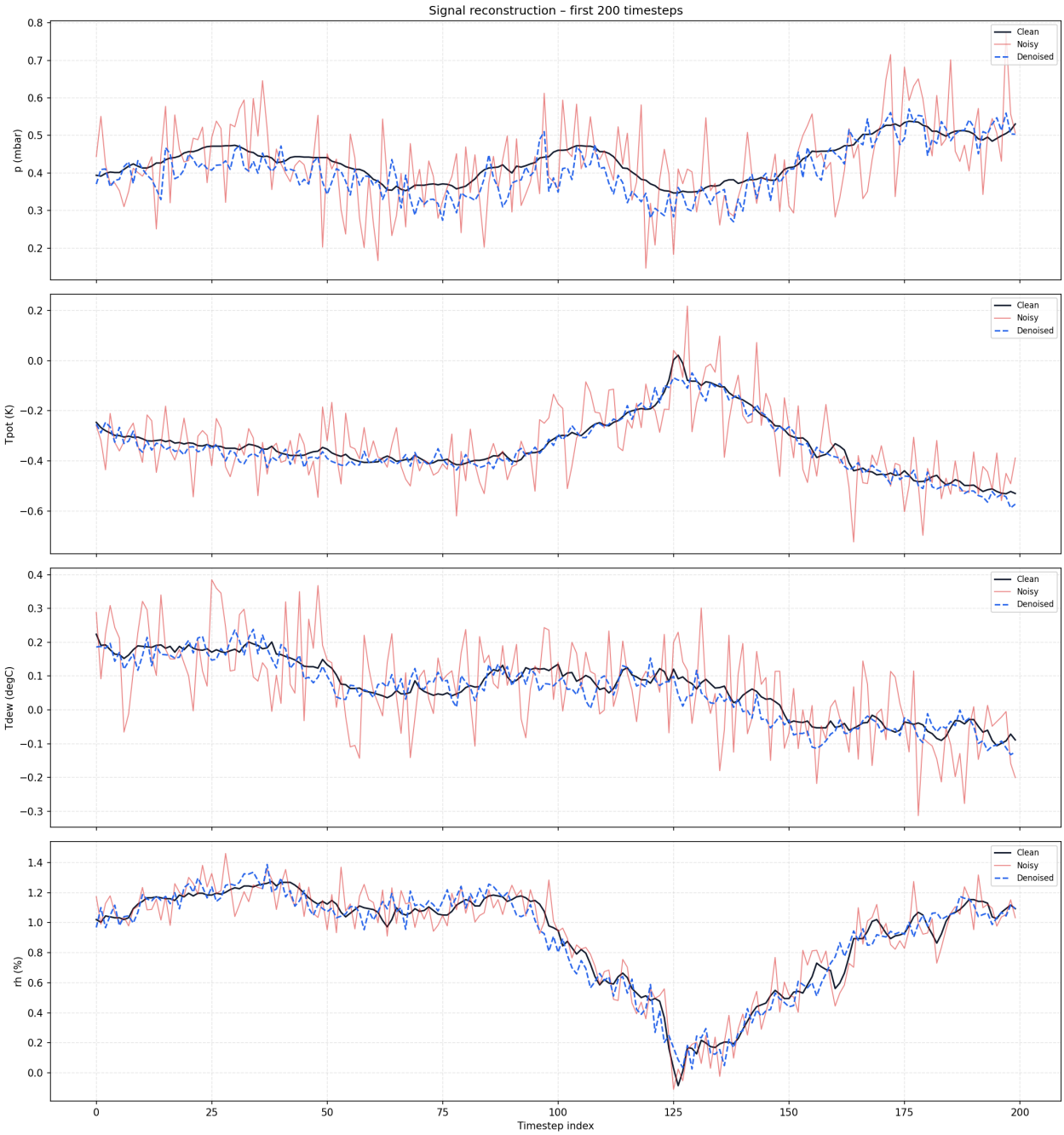

Figure 2: Four sensor features over 200 timesteps. Black = clean ground truth, red = noisy input ($\sigma = 0.1$), blue dashed = autoencoder reconstruction. The denoised signal closely follows the clean reference.

Figure 3 shows the distribution of per-sample reconstruction errors across the full test set. The tight, low-MSE distribution confirms consistent reconstruction quality.

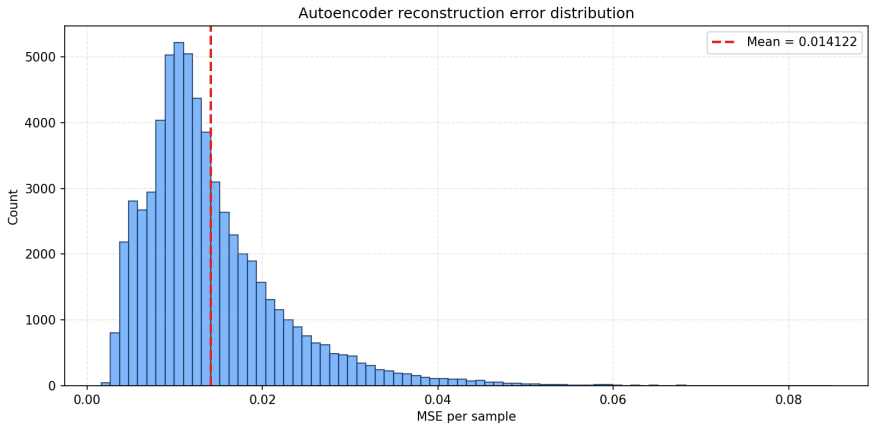

Figure 3: Histogram of per-sample MSE reconstruction error on the test set. The majority of samples have MSE below 0.025.

4.3 Classification Comparison at noise = 0.1

The Assignment 4 CNN classifier was applied to three test-set variants at $\sigma = 0.1$:

Variant	Accuracy	Precision	Recall	F1
Clean	**0.9937**	**0.9935**	**0.9930**	**0.9933**
Noisy ($\sigma = 0.1$)	0.9859	0.9842	0.9854	0.9848
Denoised ($\sigma = 0.1$)	0.9819	0.9795	0.9817	0.9806

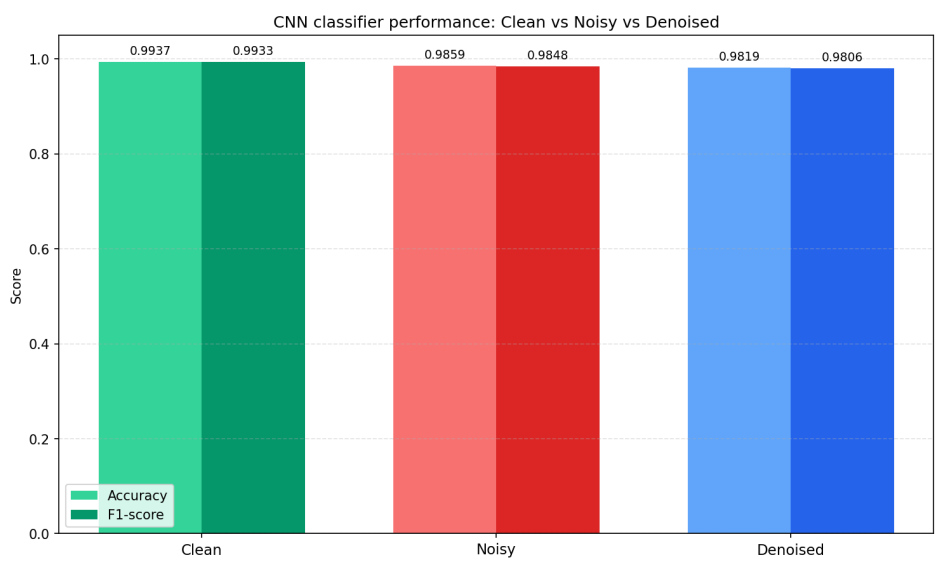

Figure 4: Accuracy and F1 comparison across the three test-set variants at $\sigma = 0.1$.

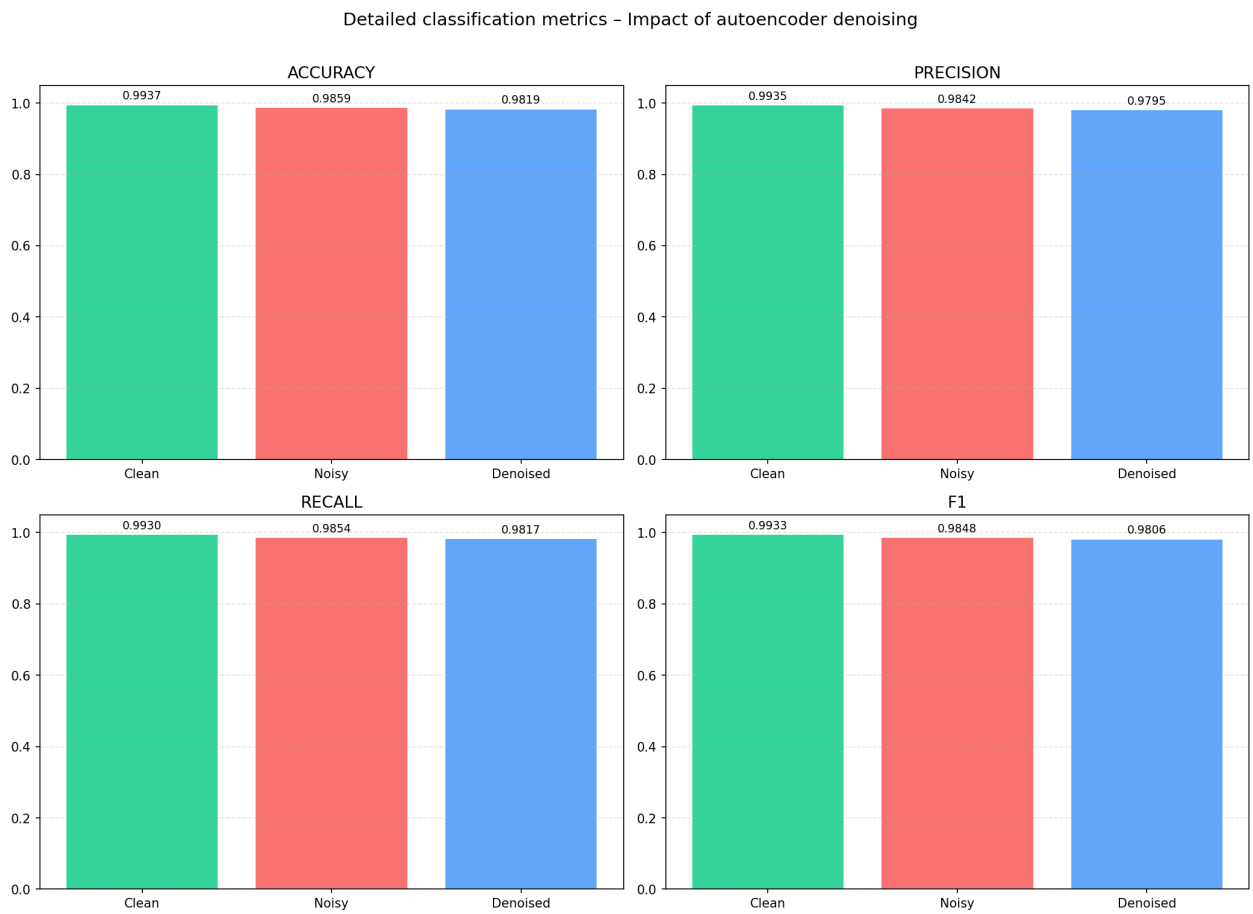

Figure 5: All four classification metrics (accuracy, precision, recall, F1) for clean, noisy, and denoised test sets at $\sigma = 0.1$.

At this low noise level the CNN is already highly robust (-0.78 pp accuracy from clean to noisy). The autoencoder's own reconstruction error slightly exceeds the noise damage, so denoised accuracy is marginally lower than noisy at this noise level. The crossover point is observed at approximately $\sigma \approx 0.15$.

4.4 Noise-Level Sweep

σ	Noisy Acc	Den. Acc	Δ Acc	Noisy F1	Den. F1	Δ F1
0.05	0.9912	0.9840	-0.72 pp	0.9905	0.9828	-0.77 pp
0.10	0.9853	0.9826	-0.27 pp	0.9843	0.9813	-0.30 pp
0.20	**0.9744**	**0.9788**	**+0.44 pp**	**0.9725**	**0.9773**	**+0.48 pp**
0.30	**0.9619**	**0.9733**	**+1.14 pp**	**0.9592**	**0.9713**	**+1.21 pp**
0.50	**0.9351**	**0.9609**	**+2.58 pp**	**0.9308**	**0.9580**	**+2.72 pp**

Bold rows indicate regimes where denoising provides a **net positive gain**. The benefit grows monotonically with noise intensity.

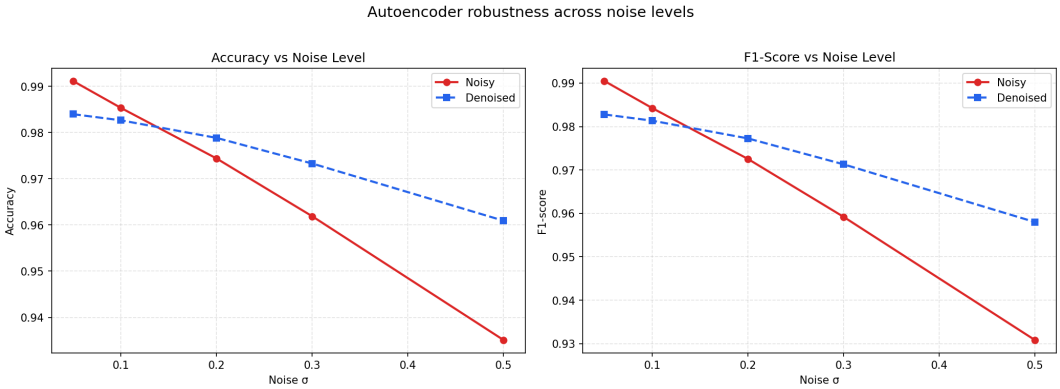

Figure 6: Accuracy and F1-score of the CNN classifier on noisy (solid) vs. denoised (dashed) test data across noise levels. The lines cross around $\sigma \approx 0.15$, after which denoising consistently improves performance.

---

5. Discussion

Why is denoising harmful at low noise? The autoencoder compresses the 624-dimensional input to a 64-dimensional bottleneck, introducing its own reconstruction error (MSE ≈ 0.0139). At $\sigma = 0.1$, the SNR is high enough for the CNN to tolerate the noise directly; the autoencoder's compression artefacts are comparably damaging. Once the noise energy exceeds the reconstruction error (around $\sigma = 0.15$), denoising becomes strictly beneficial.

Compression ratio and bottleneck choice. A 64-unit bottleneck achieves approximately 10× compression ($624 \rightarrow 64$). Reducing the bottleneck would lower reconstruction quality; increasing it would reduce the denoising effect. 64 units was selected empirically.

Practical implication. In a real-world IoT pipeline, if expected sensor noise is low ($\sigma < 0.15$ in normalised scale), the overhead of denoising may not be justified. At higher noise levels – which correspond to realistic sensor faults, transmission errors, or environmental interference – the autoencoder provides a clear and measurable benefit.

---

6. Conclusion

A symmetric fully-connected autoencoder with a 64-dimensional bottleneck successfully learns the manifold of clean Jena Climate sensor windows and serves as an effective denoising pre-processor for the downstream CNN binary classifier. Key findings:

- The autoencoder converges to a stable validation MSE of **0.013918** in approximately 15 epochs.
- At low noise ($\sigma \leq 0.1$), the CNN is inherently robust and denoising provides no benefit.
- At moderate and high noise ($\sigma \geq 0.2$), denoising **consistently improves** both accuracy and F1.
- At the highest tested noise ($\sigma = 0.5$), denoising recovers **+2.58 pp accuracy** and **+2.72 pp F1**.

These results demonstrate that autoencoders are a practical and lightweight defence against sensor noise, scaling in effectiveness with corruption intensity.

---

7. Deliverables

File	Description
assignment_5_solution.py	Full Python solution
autoencoder.h5	Saved trained autoencoder (4.9 MB)
results_summary.json	All numeric results
figures/01_ae_learning_curves.png	Train/val loss curves
figures/02_signal_comparison.png	Clean vs. noisy vs. denoised signals
figures/03_classification_comparison.png	Accuracy and F1 bar chart
figures/04_detailed_metrics.png	4-metric comparison
figures/05_reconstruction_error.png	Per-sample MSE histogram
figures/06_noise_levels.png	Accuracy and F1 across noise-level sweep
presentation.html	8-slide HTML presentation
Report.pdf	This document

References

1. Vincent, P., Larochelle, H., Bengio, Y., and Manzagol, P.A. (2008). *Extracting and composing robust features with denoising autoencoders*. ICML 2008.
2. Klambauer, G., Unterthiner, T., Mayr, A., and Hochreiter, S. (2017). *Self-normalizing neural networks*. NeurIPS 2017.
3. Chollet, F. (2021). *Deep Learning with Python*, 2nd ed. Manning Publications.
4. Dataset: Jena Climate Dataset. Max Planck Institute for Biogeochemistry. https://www.bgc-jena.mpg.de